MULTI-MODAL DEEP CONTENT END-TO-END MUSIC RECOMMENDATION

Seoyeon Park

Supervisor: Dr. Lina Yao
Co-supervisor: Dr. Junbum Kwon

School of Mathematics and Statistics
UNSW Sydney

March 2026

Submitted in partial fulfilment of the requirements of the degree of
Data Science and Decisions

Acknowledgements

I would like to thank my supervisor Lina for her continuous encouragement and for helping me build confidence throughout this project. Her support enabled me to explore the field of music recommendation in depth and with curiosity.

I am also grateful to my co-supervisor Junbum for providing steady guidance and helping me shape the overall direction of this research. His support gave me clarity and motivation at each stage.

Many thanks as well to Haowei for his practical and approachable technical advice, which played a key role in helping me move the project forward.

Through this experience, I've come to appreciate the research process both intellectually and personally. I've learned a great deal, and I'm sincerely thankful for all the support.

Seoyeon Park, May 2025.

Abstract

As music streaming platforms grow, users face choice overload, increasing the demand for personalized recommendation systems. Traditional approaches—collaborative filtering (CF) and content-based filtering (CBF)—suffer from cold-start issues or limited capacity to model user preferences. Recent studies have proposed end-to-end models that directly learn user and item embeddings from content features such as audio. A representative example is the Deep Content-User Embedding Model, which jointly learns user and item embeddings from audio content through an end-to-end architecture. However, this approach has key limitations: reliance on a single modality, high computational cost from mel-spectrogram processing, oversimplified binary feedback, and the use of scale-sensitive similarity metrics like cosine similarity.

To address these issues, we propose a deep content-based end-to-end model that leverages multimodal inputs—track names, album names, and lightweight audio features. Textual metadata is encoded using BERT and compressed via autoencoders, while features are fused through concatenation or cross-attention. User embeddings are learned from playlist identifiers, with training guided by a max-margin loss and similarity computed via Pearson correlation. Experiments on a subset of the Spotify Million Playlist Dataset show that audio features are the most informative, and combinations such as audio + track name and album name + track name yield the best performance. BERT embeddings outperform Word2Vec, and Pearson correlation provides a more robust similarity estimation than cosine similarity. A unified 128-dimensional embedding offers a balanced trade-off between expressiveness and efficiency. The model's performance is evaluated using Hit Rate@200 to assess recommendation accuracy.

Contents

Chapter 1 Introduction 1
1.1 Literature Coverage . . . 1
1.1.1 Collaborative Filtering . . . 2
1.1.2 Content-Based Filtering . . . 3
1.1.3 Deep Learning Approaches . . . 4
1.1.4 Deep content based end-to-end model . . . 4
1.1.5 Limitation . . . 4
1.2 Originality . . . 5
1.3 Paper Organization . . . 6

Chapter 2 Preliminary 7
2.0.1 BERT . . . 7
2.0.2 Autoencoder . . . 7
2.0.3 Attention . . . 8

Chapter 3 Methodology 9
3.1 Model Architecture . . . 9
3.2 User Embedding Architecture . . . 10
3.3 Item Embedding Architecture . . . 11
3.3.1 Track Embedding . . . 11
3.3.2 Album Embedding . . . 12
3.3.3 Audio Embedding . . . 13
3.4 Training Strategy . . . 14
3.4.1 End-to-End Learning for User and Item Embedding . . . 14
3.4.2 Train-Test Split . . . 16
3.4.3 Max-Margin Loss . . . 16

Chapter 4 Environment Setup 18
4.1 Dataset . . . 18
4.2 Data Preprocessing . . . 20
4.2.1 Standardisation . . . 20
4.2.2 BERT . . . 20

Chapter 5 Evaluation and Experiments 21
5.1 Evaluation . . . 21
5.2 Comparison of Similarity Measures . . . 21
5.3 Text Features Embedding Method Comparison . . . 22
5.4 Model Variants Based on Feature Combinations . . . 22
5.5 Embedding Dimension Size Comparison . . . 24

5.6 Discussion 25
5.6.1 Analysis of Similarity Measures 25
5.6.2 Analysis of Text Embedding Techniques 26
5.6.3 Analysis of Model Variants Based on Feature Combinations 26
5.6.4 Analysis of Embedding Dimension Selection 28

Chapter 6 Conclusion and Future Work 29
6.1 Conclusion 29
6.2 Future Work 30

References 33

Appendix 35

Chapter 1

Introduction

1.1 Literature Coverage

Music is often described as the art of sound. It comprises fundamental elements such as melody, harmony, rhythm, and tempo, and it evokes the listener to a wide range of emotional responses. It enables individuals to externalize feelings such as joy, sadness, or anger. Music serves as a powerful emotional expression tool that helps listeners relieve stress and enhance mental resilience and healing. Furthermore, it assists in everyday mood regulation depending on the day's mood, the weather, the time or the atmosphere of a particular moment.

With the advancement of digital technology, the way people consume music has fundamentally shifted. Rather than purchasing physical albums or downloading individual tracks, most listeners today access music through streaming platforms such as Spotify, YouTube Music, and Apple Music simply using their smartphones, smart speakers, or laptops.

The convenient accessibility of music streaming services enables listeners to have easy access to a vast number of songs. While this accessibility enhances the personal development of musical preferences, it has also introduced the new challenge called choice overload. Among millions of available musics, discovering music that aligns with a user's preference or mood is demanding. Therefore, to improve the user's platform experience by providing preference-matched music and to engage users, the intelligent, sophisticatedly curated, personalized music recommendation system has become an essential component of streaming platforms. These systems help users navigate vast song libraries, reduce cognitive load, and improve satisfaction by providing content tailored to individual tastes and listening behaviors.

Various recommendation algorithms have been studied to address the growing demand for music recommendation systems. These recommendation systems can be broadly classified into two primary categories, item-based and content-based, depending on whether they rely on user interaction history or item attributes respectfully. Collaborative filtering (CF) is a well-known item-based algorithm that recommends music by utilizing similar users' preferences. In contrast, content-based filtering (CBF) relies on items' inherent features based on the user's previous listening history. Deep learning approaches have emerged more recently to overcome the limitations of traditional methods by learning richer, non-linear representations of both users and items. We introduce each methodology, detailing its mechanism,

advantages and limitations in the following sections.

1.1.1 Collaborative Filtering

Collaborative Filtering (CF) uses a matrix to capture the user's listening behavior for each item, which, in our case, is music. Typically, we represent each user's preference for items in a user-item matrix, where rows correspond to users, columns to music tracks, and values to user feedback such as likes or ratings. Based on this matrix, CF measures similarity between users or items by computing metrics such as cosine similarity or the Pearson correlation coefficient and recommends songs based on these user-item interactions.

In general, CF systems can be categorized as memory-based and model-based. The memory-based method focuses on predicting a target user's behavior toward candidate music, such as whether user1 would like or dislike song1, based on similar users or similar items. This type is further divided into user-based filtering and item-based filtering. User-based filtering provides recommendations based on the target user's past behavior compared to other users. If the user's behavior is similar to that of specific users, the system assumes the user will respond similarly to unseen music. In contrast, item-based filtering recommends music based on the user's interaction with similar songs. For instance, if a user frequently listens to a particular song, and many other users who listened to that song also often listen to another specific song, the system will likely recommend that second song. Model-based methods use machine learning techniques to learn patterns from interaction data and make predictions [10].

While CF effectively captures collective trends and shared preferences across users, it tends to be less personalized than methods that incorporate the content or context of an item. CF does not consider the content of the music, such as its meaning, emotion, or audio characteristics. Instead, it solely relies on user behavior patterns. As a result, CF does not truly understand the individual user. Instead, it assumes that users who behaved similarly in the past will share similar preferences in the future, which may not always be true, as users with similar listening histories may consume music due to different contextual or content preferences.

Furthermore, CF suffers from the cold start problem, which refers to the challenge of making accurate recommendations for new users or items that lack sufficient historical interaction data. Since CF heavily relies on historical user interactions, new users with little to no listening history cannot receive personalized recommendations. Similarly, newly released songs suffer from a lack of user feedback, making it difficult for the system to assess their relevance to specific users. In addition, CF often favors popular songs over lesser-known tracks due to data sparsity and algorithmic bias, ultimately reducing diversity in recommendations [17].

1.1.2 Content-Based Filtering

Content-Based Filtering (CBF) is an information retrieval approach that recommends items to users based on the features of individual items [11]. In the context of music recommendation, CBF builds an item profile for each song using internal characteristics (e.g., tempo, energy, danceability) and descriptive metadata (e.g., artist name, album name). A user profile is constructed based on the user's past listening history and expressed preferences, such as likes, dislikes, or ratings. The recommendation system compares user and item profiles, often using similarity measures such as cosine similarity, Euclidean distance, or Pearson correlation—to estimate the user's potential interest in unseen music.

CBF often employs deep learning models or machine learning techniques to predict user behavior, such as ratings or likes, using item attributes as predictors. For example, if a user frequently listens to R&B songs that feature moderate tempo, smooth rhythm, and high valence, with hopeful track names, the system may recommend other songs with similar audio characteristics and emotional tone. This allows CBF to generate personalized recommendations solely based on item content, making it particularly effective in scenarios with limited user-item interaction data.

While collaborative Filtering (CF) recommends music based on the preferences of users with similar tastes, CBF identifies music that share similar features with items users liked in the past. This makes CBF a more personalized approach, as recommendations are tailored directly to an individual's preferences not relying on broader user-item interactions [5]. One key advantage of CBF is its ability to mitigate the item cold start problem. Since recommendations are based on content attributes rather than user feedback, newly released songs can still be recommended if their features align with existing music in the system. Furthermore, since CBF relies on the contents of items, not the popularity of songs, it reduces algorithmic bias [17].

However, the effectiveness of CBF is largely dependent on the quality of feature extraction. In this context, feature extraction refers to deriving meaningful music representations from raw data such as audio signals or textual metadata. This includes acoustic attributes (e.g., tempo, energy, valence) and semantic features (e.g., track or album names), which form the basis of recommendation decisions. Because content-based recommendations rely solely on these item features, inadequate or uninformative representations can limit the system's ability to identify music that aligns with user preferences.

Obtaining high-quality feature representations for all songs presents a challenge due to high computational complexity. This makes it difficult to capture the full complexity of music, particularly in audio or text-based recommendations. As a result, deep learning–based approaches are often employed to learn more meaningful representations of music [17].

1.1.3 Deep Learning Approaches

Deep learning approaches have significantly enhanced the capacity of content-based filtering (CBF) systems by addressing two significant challenges: effective feature extraction and learning complex user-item relationships.

First, deep learning models have been widely adopted for extracting high-level semantic features from raw audio signals or textual metadata. For instance, in this study, BERT-based pretrained models are used to derive contextual embeddings from album and track names, while an autoencoder is applied to reduce dimensionality and retain essential information.

More importantly, deep learning enables the system to model the nuanced relationship between item characteristics and user preferences during training. Instead of relying on predefined similarity measures, the network learns to predict the relevance between user and item embeddings through a fully connected layer. The model thereby captures individual preference patterns by learning from user interaction signals rather than manually designed rules.

As a result, deep learning–based approaches are often employed to learn more expressive and personalized representations of music, particularly in scenarios where subtle and complex interactions with content features such as audio and text features shape user preferences.

1.1.4 Deep content based end-to-end model

Hybrid approaches, which integrate multiple modalities (e.g. user-item interactions, content of item) into one system have been proposed to leverage the complementary strengths of CF and CBF. CF captures common consumption patterns or item popularity while CBF can address the cold start problem and provides more personalized recommendations. However, these approaches separate the training phases of CF and CBF models which may lead to suboptimal solutions. To mitigate this, an end-to-end model which jointly learns user and item representations has been explored. Lee (2018) proposed a deep content-user embedding model in an end-to-end manner that combines user-item interaction signals with audio-based item content using convolutional neural networks [8]. This end-to-end approach eliminates the need for separate training phases and improves recommendation performance. The proposed model achieves 0.7914 AUC in the recommendation task. Meanwhile, the hybrid method (WMF+Regression), which requires separate training phases, achieves 0.6967 AUC. This demonstrates the effectiveness of the end-to-end approach compared to the traditional hybrid approach.

1.1.5 Limitation

While the deep content-user embedding music recommendation model is effective in music recommendation and auto-tagging, the baseline architecture can be improved in generalization and computation.

First, the model is restricted to a single modality. The independent audio content constrains the expressive capacity of the model and may not be sufficient to capture the nuanced characteristics of musical content. The absence of multimodality limits the model's robustness, particularly in scenarios with sparse item features or text features that are significant in capturing the mood of music.

Second, the model employs cosine similarity to compute the relevance between user and item embeddings. While this approach is widely used due to its computational simplicity and efficiency, it fails to account for the mean-centered relationships between embeddings. In particular, cosine similarity is limited in capturing the directional consistency of user preferences when the magnitude and baseline levels of embeddings differ. This may lead to suboptimal matching, especially in settings where relational patterns are more important than vector orientation.

Third, the spectrogram-based representations in prior works introduce substantial computational overhead. Mel-spectrogram extraction and CNN-based processing demand significant resources and time, posing scalability challenges for large-scale deployment in large-scale recommendation systems.

Fourth, the binary treatment of implicit feedback, where any interaction is considered positive, oversimplifies the complexity of user behavior. Previous models treated any song with a play count of one or more as a positive interaction without considering how frequently the user listened. This means that a song played once and dozens of times are treated equally, even though they may reflect substantially different levels of user preference. As a result, the model may misinterpret noisy or low-signal interactions as strong indicators of user interest, leading to degraded recommendation quality.

1.2 Originality

Building on this framework, our study proposes an enhanced content-based recommendation model incorporating multi-modal features, including audio, track name, and album name metadata. Unlike previous approaches that treat each content source independently or rely on a single modality, our model fuses these heterogeneous inputs using a cross-attention mechanism to learn a unified, semantically rich item representation. We adopt BERT in our model to represent musical metadata track and album names in a context-aware manner, improving personalization. In addition, we apply autoencoders to compress BERT-based textual embeddings, reducing dimensionality and extracting essential information.

In addition, we adopt the Pearson correlation coefficient instead of cosine similarity to compute the similarity between user and item embeddings. While cosine similarity measures the angle between two vectors, it ignores differences in baseline preferences. In contrast, Pearson correlation captures relative variations by centering each vector, allowing the model to reflect better whether users and items follow similar patterns of preference.

Furthermore, we use a lightweight set of 11 numerical audio features—such as danceability and energy—obtained through the Spotify API rather than relying on computationally expensive spectrogram representations. These features are combined with metadata to construct a unified data frame that serves as input to the model, improving efficiency without sacrificing representational power.

Lastly, we use playlist inclusion as a binary user preference indicator instead of simply checking whether a song was played. In previous approaches, a song was considered positive if it had been played at least once, regardless of whether it was played briefly or accidentally. In contrast, we treat a song as a positive signal only when explicitly included in a user's playlist. This provides a more intentional and reliable indicator of preference, as playlist additions are more likely to reflect deliberate user choices rather than incidental or passive listening behavior.

The entire model is trained in an end-to-end fashion, jointly optimizing user and item embeddings to capture both behavioral patterns and deep content semantics. This unified design directly addresses key limitations of previous models, particularly with respect to modality reliance, similarity computation, feedback signal accuracy, and computational efficiency.

1.3 Paper Organization

This thesis is organized as follows. Chapter 1 introduces the background and motivation of the study, outlines the limitations of existing recommendation approaches, and highlights the originality of the proposed method. Chapter 2 presents preliminary concepts, including BERT, autoencoders, and attention mechanisms, which serve as the foundation for the proposed model. Chapter 3 describes the methodology in detail, including the overall model architecture, user and item embedding design, and the training strategy. Chapter 4 outlines the experimental environment, covering the dataset used and the pre-processing steps. Chapter 5 presents the evaluation results along with discussion and analysis. It compares similarity measures and text embedding techniques, examines the effect of different feature combinations, and analyzes embedding dimension sizes. Finally, Chapter 6 summarizes the main findings and suggests directions for future research.

Chapter 2

Preliminary

2.0.1 BERT

In music recommendation tasks, textual metadata such as album or track names often carry semantic signals. Prior studies have shown that language model pre-training significantly improves performance across various NLP tasks [14] . However, traditional unidirectional language models such as left-to-right or right-to-left models are limited in their ability to fully capture the contextual information [16].

To overcome these limitations, BERT (Bidirectional Encoder Representations from Transformers) was introduced as a context-aware language model developed by Google. It is pre-trained on large-scale unlabeled corpora and is designed to capture deep bidirectional contextual representations by jointly conditioning on both left and right context. Unlike Word2Vec, which produces static embeddings based on limited local context, BERT considers the full sentence context for each token. This enables BERT to generate richer and more semantically meaningful embeddings, particularly when the same word appears in different contexts [6].

This makes BERT particularly effective for music recommendation tasks, where textual metadata such as track and album names often contain short, ambiguous, or context-dependent information.

2.0.2 Autoencoder

Multi-head attention models such as BERT increase the number of attention heads to improve expressive power, which leads to low rank bottleneck problems where attention heads fail to capture sufficient context due to limited representational capacity. BERT adopts high-dimensional embeddings to address this, enabling better attention performance across multiple heads. However, this causes an increase in the computational cost, memory consumption and model size [2]. In the context of music recommendation, where BERT embeddings are used as input features, reducing the dimensionality becomes essential for building a scalable and efficient model.

Autoencoders are neural networks designed to learn efficient and compact latent representations of input data by encoding it into a compressed latent space and then reconstructing the original input from this lower-dimensional representation. They are commonly used for dimensionality reduction when the number of original input nodes exceeds the number of hidden layer nodes [15].

In recent work, Laskar (2022) demonstrated that autoencoders can effectively compress pre-trained BERT embeddings [7]. The compressed embeddings preserve comparable performance to the original embeddings in downstream tasks, while significantly reducing computational cost and storage requirements. Moreover, autoencoders can extract salient features by learning useful latent representations from high-dimensional input data [7].

2.0.3 Attention

Feature fusion refers to the selection and combination of relevant features. Its importance becomes crucial with increases in noise and dimensionality of the feature set. It eliminates redundant and irrelevant features offering a better feature set in terms of performance and computational cost. Furthermore, it facilitates the integration of multiple features from different modalities constructing more informative and compact representations [9].

While simple fusion strategies such as addition or concatenation are widely adopted for their simplicity, they often fail to capture the complex semantic relationships among heterogeneous features. This limitation becomes especially critical in multimodal systems, where features from different modalities [4].

Attention mechanisms introduced by Vaswani (2017) have gained popularity as a way to selectively weigh information based on its contextual relevance [14]. Attention-based fusion approaches have been proposed to dynamically assign importance weights to each feature, enabling more context-aware and informative integration. For example, the Attentional Feature Fusion (AFF) module proposed by Dai (2020) improves representational quality by adaptively combining features with respect to both local and global channel-wise statistics [4].That is, the model considers both fine-grained variations within each feature map (local) and the overall distribution across channels (global) when determining fusion weights.

To further capture the interactions between multiple modalities, cross-attention can be employed. Cross-attention is an attention mechanism that allows the model to focus on relevant information in one modality by conditioning on another. It operates by projecting one modality (e.g., audio) into a set of query vectors, while another modality (e.g., track name) is projected into key and value vectors. The attention weights are calculated by matching queries to keys, and are used to aggregate the values into a new, context-aware representation. This enables the model to learn and emphasize inter-modal relationships—meaningful associations between distinct types of information, such as acoustic features and textual metadata [13]. For instance, a track name may imply a particular genre or mood that aligns with its sound characteristics.

Chapter 3

Methodology

3.1 Model Architecture

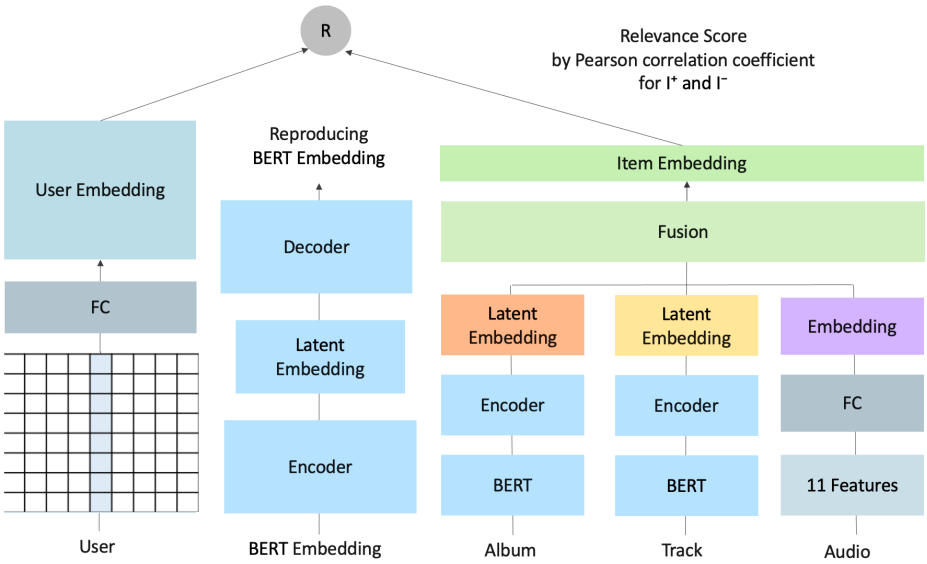


Figure 3.1: Multi-modal Deep Content based end-to-end model architecture

An overview of the proposed deep content end-to-end recommendation model is presented in Figure 3.1. The architecture is designed to generate user and item embeddings from multimodal content features in an end-to-end manner. The model consists of two main components: a user embedding and an item embedding. The user embedding is derived from a playlist-level representation, while the item embedding integrates features from track names, album names, and audio attributes. To capture the semantic meaning of textual metadata, pre-trained BERT embeddings are employed, which are further compressed using autoencoders. Audio features are processed through a fully connected layer. The resulting embeddings from each modality are either concatenated or fused via cross-attention to form a unified item representation. These embeddings are then used to compute the relevance score between users and items using the Pearson correlation coefficient. The relevance scores are calculated for both I^+ and I^-, which represent positive and negative item sets, respectively. Then a maximum-margin loss is computed on the basis of these relevance scores. More details are discussed in the following sections.

3.2 User Embedding Architecture

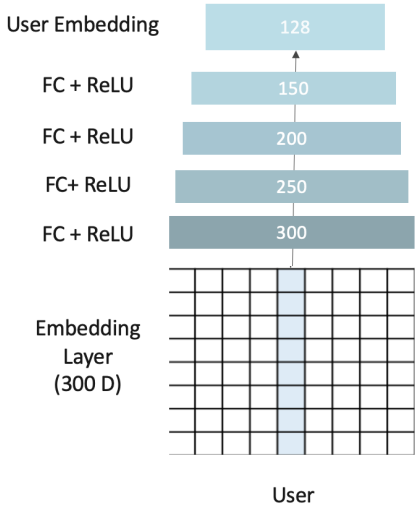


Figure 3.2: User embedding model architecture

Each playlist is first represented as a 300-dimensional learnable embedding vector. To project this into a compact latent space, we apply a series of fully connected layers with gradually decreasing dimensionality: $300 \rightarrow 250 \rightarrow 200 \rightarrow 150 \rightarrow 128$. Between each linear transformation, we insert ReLU activation functions to introduce non-linearity, allowing the network to learn more expressive representations beyond simple linear mappings.

The ReLU function is defined as:

$$\mathrm{ReLU}(x) = \max(0, x)$$

It is chosen for its computational efficiency, robustness against the vanishing gradient problem, and ability to promote sparsity by deactivating negative units. Specifically, ReLU enables the model to maintain strong gradient signals during backpropagation for positive activations, which facilitates more stable and faster convergence during training. Moreover, by zeroing out negative values, ReLU induces sparsity in intermediate representations, which helps reduce representational redundancy and computational overhead. This sparsity improves model efficiency and contributes to better generalization by discouraging overfitting. As a result, ReLU is particularly well-suited for deep architectures that demand expressive power and computational efficiency.

This multi-layer projection reduces dimensionality from 300 to 128 and gradually enables the model to learn a meaningful user representation from the initialized embedding. The model captures semantic patterns in user preferences through this

transformation, producing a 128-dimensional embedding suitable for computing relevance with item representations.

3.3 Item Embedding Architecture

3.3.1 Track Embedding

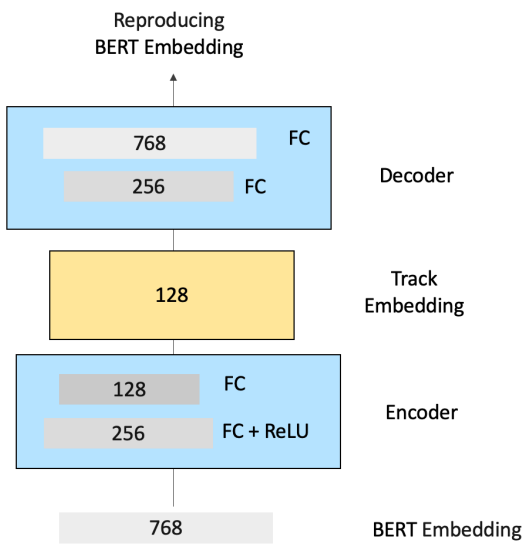


Figure 3.3: Architecture of the track embedding module

The track name representations are obtained using the [CLS] token output from a pre-trained BERT model, capturing contextualized semantic information from the input text. These 768-dimensional BERT embeddings, used without further fine-tuning, are passed into an encoder network to compress the input into a lower-dimensional representation. For more details on how the BERT embeddings are constructed and processed, refer to Section 4.2.2.

The encoder consists of two fully connected layers with a ReLU activation function in between, reducing the dimensionality from 768 to 128. The ReLU activation introduces non-linearity and promotes sparsity in the intermediate representation, enabling the encoder to learn more abstract and informative features while helping to prevent overfitting. For this purpose, we use only the encoder component to reduce dimensionality and extract useful semantic features.

The resulting 128-dimensional track embedding preserves essential semantic features from the original BERT representation and is subsequently used as a component of the item embedding in the fusion process.

3.3.2 Album Embedding

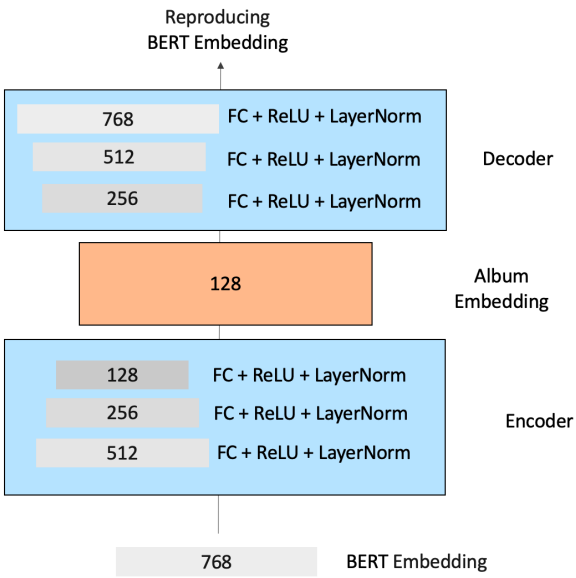


Figure 3.4: Architecture of the album embedding module

We used a pre-trained BERT model to obtain 768-dimensional contextualized embeddings that capture the semantic meaning of album names. These embeddings were passed through an encoder module for dimensionality reduction, where only the encoder of an autoencoder was used. The encoder is a three-layer feedforward network that compresses the original input from 768 to 512, then to 256, and finally to a 128-dimensional latent representation. A ReLU activation and Layer Normalization follow each linear layer to stabilize the training dynamics. The use of ReLU introduces non-linearity, helping the model capture complex patterns.

Deep learning models often exhibit strong learning capabilities but typically require long training times and suffer from slow convergence. Normalization techniques have been widely adopted to improve training efficiency and stability. Layer Normalization (LN) normalizes the input features within each individual sample, making it especially suitable when sample-wise variation is high. In contrast, Batch Normalization (BN) normalizes features across samples in a batch, which introduces a dependency between training examples and can be sensitive to batch size [1]. Thus, we apply Layer Normalization instead of Batch Normalization, as LN operates independently for each input sample and does not rely on batch-level statistics.

Inputs such as track names and album names tend to be highly unique and diverse across samples, often lacking a consistent statistical distribution. This irregularity may result in internal covariate shift, where the distribution of inputs shifts across different training examples, making optimization more difficult. To mitigate this issue, Layer Normalization normalizes each intermediate representation within a

single sample by computing the mean and variance across its feature dimensions:

$$\mathrm{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad \mu = \frac{1}{H}\sum_{i=1}^{H} x_i, \quad \sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2 \tag{3.3.1}$$

Here, $x \in \mathbb{R}^H$ is the feature vector within a sample, μ and σ^2 denote the mean and variance computed across the H dimensions of x, and ϵ is a small constant added for numerical stability.

By normalizing the feature values within each sample to have a stable distribution (i.e., zero mean and unit variance), Layer Normalization helps ensure that every layer receives inputs with consistent properties. This results in faster and more stable training while also making the model robust to batch size, which is beneficial when handling highly variable inputs that lack shared distributional patterns.

3.3.3 Audio Embedding

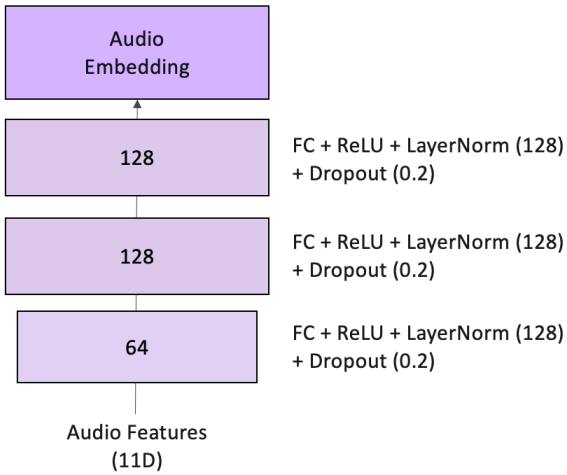


Figure 3.5: Architecture of the audio embedding module

We design a feedforward neural network tailored for audio embedding to transform raw audio features into meaningful, dense representations. The input to this module consists of 11-dimensional predefined audio features, such as danceability, energy, and tempo, extracted from the Spotify API. These features are described in more detail in Section 4.1. This input is first projected to a higher-dimensional space through a series of fully connected layers: $11 \rightarrow 64 \rightarrow 128 \rightarrow 128$. After each linear transformation, a ReLU activation introduces non-linearity, followed by layer normalization to stabilize the training and mitigate internal covariate shifts —i.e., shifts in the distribution of layer inputs during training. A dropout rate 0.2 is applied after each normalization step to reduce overfitting.

3.4 Training Strategy

3.4.1 End-to-End Learning for User and Item Embedding

In this work, we train the model using an end-to-end approach, which jointly learns user and item embeddings in a single training phase. The `UserItemEmbedding` module integrates both user and item embeddings. User embeddings are assigned to playlist IDs and are optimized via interaction signals during training, while item embeddings are constructed from multimodal content features. This joint structure enables simultaneous optimization through a shared loss function.

The specific structure is as follows:

Construction of User Embeddings from Playlist IDs

In our model, each playlist is treated as a proxy for an individual user. A unique trainable embedding vector is assigned to every playlist ID, serving as the user representation. These vectors are retrieved during the forward pass and updated throughout training using the user embedding module. Although playlist IDs themselves carry no semantic information, the associated embeddings are refined during training by interacting with item embeddings through a max-margin loss.

This end-to-end optimization strategy enables the model to learn personalized user embeddings based solely on interaction patterns without relying on explicit user metadata, such as age or gender, or on summary features derived from past user activity, such as play counts, genre preferences, or listening durations. As a result, the user embeddings progressively capture latent preferences embedded within playlists, guided by training signals that promote alignment with items included in the same playlist and separation from unrelated ones.

Cross-Attention-Based Construction of Item Embeddings

The Item embeddings is constructed from content-based features that are audio features, album names, and track names. To effectively combine these heterogeneous modalities, we employ a cross-attention-based fusion architecture.

The audio, album, and track embeddings are first projected into a shared 128-dimensional latent space, as described in Section 3.3. Subsequently, cross-attention layers selectively aggregate information by dynamically weighting features across modalities, allowing the model to focus on the most relevant signals.

First, the album embedding attends to the track embedding via a multi-head cross-attention mechanism, allowing the album representation to incorporate track-level semantic information:

$$\mathbf{H}_{\text{album-track}} = \text{CrossAttention}(\text{query} = \mathbf{Album}, \text{key} = \mathbf{Track}, \text{value} = \mathbf{Track})$$

Subsequently, the audio embedding attends to the album-track attended representation to further integrate aligned semantic signals across audio, album, and track

modalities:

$$\mathbf{H}_{\text{audio-album-track}} = \text{CrossAttention}(\text{query} = \mathbf{Audio}, \text{key} = \mathbf{H}_{\text{album-track}}, \text{value} = \mathbf{H}_{\text{album-track}})$$

Each cross-attention block employs a multi-head attention mechanism, where the attention operation for each head formula is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where Q, K, and V denote the query, key, and value matrices, and d_k represents the dimension of the key vectors.

To enhance representational capacity, we apply multi-head attention, projecting the input into multiple subspaces and attending independently to different aspects. The outputs of all heads are concatenated and linearly transformed:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

where each head is computed as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

and W_i^Q, W_i^K, W_i^V, and W^O are learnable projection matrices.

As illustrated in Figure 3.6, each attention head computes scaled dot-product attention independently, and their outputs are concatenated and linearly projected to form the final multi-head attention representation.

In our implementation, we use four attention heads ($h = 4$), allowing the model to capture diverse relational patterns across different modalities. Each head focuses on different aspects of feature interactions, leading to richer and more robust item representations. The final fused representation is passed through a two-layer feedforward network with ReLU activation to produce a 128-dimensional item embedding.

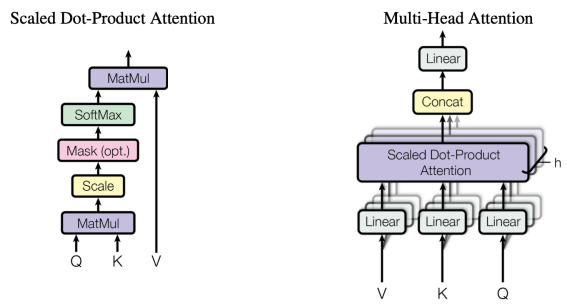


Figure 3.6: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel [14].

3.4.2 Train-Test Split

The dataset consists of 101 playlists, including 100 songs in each. We split the songs in each playlist to create training, validation, and test data sets with a 0.7, 0.15 and 0.15 ratio, respectively. The division was performed independently within each playlist, ensuring consistency in the playlist context across training and evaluation. Therefore, 70 songs were used for training, 15 for validation, and 15 for the test data set for each playlist.

3.4.3 Max-Margin Loss

Positive and Negative Sampling

During training and validation, we construct balanced positive and negative samples per playlist using the first 85 songs, excluding the final 15 tracks reserved for testing. No sampling is performed for test data. Since the evaluation relies on ground-truth ranking over the full item pool, the model assigns relevance scores to all candidate tracks, and recommendation performance is measured by how highly the ground-truth songs (i.e., the 15 held-out songs) are ranked. This allows for a direct assessment of the model's ranking ability without the need for negative sample.

Positive interactions (I^+) are defined as all observed playlist–track pairs, i.e., each playlist is paired with the 85 songs explicitly included in it (comprising 70 training and 15 validation samples).

Negative interactions (I^-) are generated by randomly sampling 85 tracks per playlist from the set of tracks *not* included in that playlist. The sampling is done without replacement from the complement set defined over the global track pool.

As a result, $I^+{}_{\text{train}}$ and $I^-{}_{\text{train}}$ each contain 7070 interactions (70 per playlist $\times$ 101 playlists), while $I^+{}_{\text{val}}$ and $I^+{}_{\text{val}}$ each consist of 1515 interactions (15 per playlist $\times$ 101 playlists). The construction of the feature set used in training and sampling is detailed in chapter 2.1. This sampling strategy is designed to reflect realistic user preferences and non-preferences, enabling the model to learn to distinguish relevant items from irrelevant ones.

Similarity Computation

To measure the relevance between a user and candidate items, we employ the Pearson correlation coefficient as our similarity function. This choice is motivated by the nature of our data and recommendation objective: instead of comparing raw embedding values directly, we aim to capture how user and item representations co-vary relative to their own mean centers

In our case, each user corresponds to a playlist, and different playlists may contain songs with diverse distributions. As a result, user and item embeddings can have different mean centers. Pearson similarity normalizes both embeddings by subtracting their respective means, aligning them to a common scale and allowing comparison of relative variation. This property is especially useful for personalized

music recommendation, where preferences may vary in intensity but share similar co-variation patterns.

Formally, given a user embedding $\mathbf{U} \in \mathbb{R}^d$ and an item embedding $\mathbf{I} \in \mathbb{R}^d$, the Pearson similarity is computed as:

$$\text{Pearson}(\mathbf{U}, \mathbf{I}) = \frac{\sum_{i=1}^{d}(U_i - \bar{U})(I_i - \bar{I})}{\sqrt{\sum_{i=1}^{d}(U_i - \bar{U})^2} \cdot \sqrt{\sum_{i=1}^{d}(I_i - \bar{I})^2}}$$

This formulation ensures *translation invariance*, meaning that the similarity measure is unaffected by shifts in the mean of each vector.

In contrast, cosine similarity is computed as:

$$\text{Cosine}(\mathbf{U}, \mathbf{I}) = \frac{\mathbf{U} \cdot \mathbf{I}}{\|\mathbf{U}\| \cdot \|\mathbf{I}\|}$$

Cosine similarity does not normalize for the mean and therefore cannot account for baseline shifts across playlists. While it is invariant to the overall vector magnitude, it is sensitive to feature-wise scale differences—a disproportionately large value in one dimension can dominate the vector's direction and distort similarity estimation. As a result, it may overlook important co-variation patterns in the embedding space that are essential for nuanced, content-based recommendation. Therefore, we adopt Pearson correlation coefficient as a similarity computation.

In our model, each training instance consists of a user $\mathbf{U}$, a positive item $\mathbf{I}^+$, and a negative item $\mathbf{I}^-$. We compute the corresponding similarity scores as follows:

$$s^+ = \text{sim}(\mathbf{U}, \mathbf{I}^+), \quad s^- = \text{sim}(\mathbf{U}, \mathbf{I}^-)$$

These similarity scores are treated as *relevance scores* and serve as direct inputs to our learning objective, which is designed to increase the score of $\mathbf{I}^+$ relative to $\mathbf{I}^-$. The formulation of this max-margin ranking loss is described in the following section.

Loss Formulation

To learn a user's preference over items, we adopt a max-margin ranking loss, which encourages the similarity score of a positive item $\mathbf{I}^+$ to be higher than that of a negative item $\mathbf{I}^-$ by at least a fixed margin. This loss is defined as:

$$\mathcal{L} = \max(0,\, m - s^+ + s^-)$$

where $s^+ = \text{sim}(\mathbf{U}, \mathbf{I}^+)$, $s^- = \text{sim}(\mathbf{U}, \mathbf{I}^-)$, and $m > 0$ is a predefined margin hyperparameter. We set the margin $m = 0.2$. The model is trained using the Adam optimizer with a learning rate of 0.001.

Chapter 4

Environment Setup

4.1 Dataset

The Spotify Million Playlist Dataset (MPD)[3] was selected as the primary dataset. The MPD contains one million user-generated playlists including playlist id, track titles, and album titles created by users on the Spotify platform between January 2010 and October 2017. In this work, we randomly select 101 playlists, each containing 100 songs. Each playlist in the MPD includes a playlist ID and a track names and album names and associated metadata such as track URIs.

To enrich the dataset with audio-based features, we used the Spotify Web API to extract acoustic characteristics for each track. Specifically, we utilized the audio_features endpoint provided by the Spotipy Python client, which returns 11 predefined audio descriptors: acousticness, danceability, energy, instrumentalness, liveness, loudness, speechiness, valence, tempo, mode, and key.

Because the API restricts requests to a maximum of 100 tracks at a time, we implemented a batching process that grouped track URIs into chunks of 100. These batches were then sent iteratively to the API to retrieve the corresponding audio features. Each response returned a list of JSON objects containing the audio data, which we parsed and converted into a pandas DataFrame.

To ensure complete retrieval, the function included basic error handling if a request failed (e.g., due to rate limits or network issues), it would wait momentarily before retrying. Finally, the collected audio features were merged with the original playlist and track metadata using aligned indexing, resulting in a complete dataset that includes both user-generated and audio-derived content.

The final feature set consists of:

- **Playlist ID** : a unique identifier assigned to each playlist. This ID is used to map user-specific preferences and is treated as the basis for generating user embeddings.

- **Text metadata**:
  - **Album name**: the title of the album associated with each track
  - **Track name**: the title of the song

- **Audio features**:
  - **Danceability**: indicates how well a track is suited for dancing, based on factors such as rhythm stability, beat strength, and tempo. Scores range from 0.0 (least danceable) to 1.0 (most danceable)

  - **Energy**: represents the perceived intensity and activity level of a track, ranging from 0.0 to 1.0. Tracks with high energy tend to sound fast, loud, and dynamic, while those with low energy feel more subdued or calm.

  - **Key**: denotes the musical key in which a track is composed, using integer values that correspond to pitch classes (e.g., C = 0, D = 2). If the key cannot be determined, a value of -1 is returned.

  - **Loudness**: measures the overall sound level of a track, expressed in decibels (dB). This value is averaged over the full duration of the song and typically falls between -60 dB (very quiet) and 0 dB (very loud)

  - **Mode**: refers to whether the track is in a major or minor key. A value of 1 indicates a major mode, while 0 represents a minor mode.

  - **Speechiness**: assesses the presence of spoken words within a track. Values approaching 1.0 suggest that the track is primarily spoken (e.g., a podcast), whereas lower values indicate that the content is more musical.

  - **Acousticness**: measures the likelihood that a track is acoustic, with values ranging from 0.0 to 1.0. Higher values indicate stronger confidence that the song contains primarily acoustic elements

  - **Instrumentalness**: estimates the extent to which a track contains no vocals. A score close to 1.0 suggests that the track is likely instrumental. Values above 0.5 generally indicate a high probability of being vocal-free

  - **Liveness**: indicates the presence of a live audience or live performance context. Tracks with a score above 0.8 are considered highly likely to be recorded live

  - **Valence**: describes the emotional positivity conveyed by the track, on a scale from 0.0 to 1.0. High valence values are associated with cheerful, happy-sounding music, while lower values suggest more somber or tense moods

  - **Tempo**: represents the estimated speed of the track in beats per minute (BPM). It reflects the rhythmic pace or beat rate of the music.

4.2 Data Preprocessing

4.2.1 Standardisation

To ensure consistent feature scaling, we applied standardization to all audio features using scikit-learn's StandardScaler. This method transforms each feature to have zero mean and unit variance, preventing any single feature—such as tempo or loudness, which have relatively large numeric ranges—from dominating others like energy or valence, which are constrained between 0 and 1. It improves training stability and ensures that the model treats each feature on a comparable scale during learning.

4.2.2 BERT

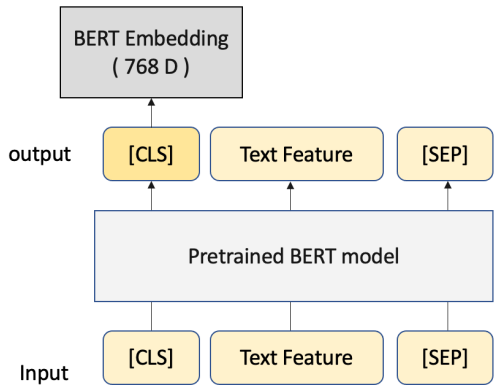


Figure 4.1: Track and album name embedding using BERT [CLS] output

We used the pre-trained BERT-base-uncased model from the Hugging Face Transformers library to generate semantic representations of track and album names. Each input string was tokenized using the BERT WordPiece tokenizer, with special tokens [CLS] added at the beginning and [SEP] at the end of the sequence and padded to a fixed length of 10 tokens for uniformity. BERT uses the [CLS] token to aggregate sentence-level information, while [SEP] marks the end of the input sequence. The tokenized input was passed through the BERT model in evaluation mode, without parameter updates.

We extracted the final hidden state corresponding to the [CLS] token, which is designed to aggregate sentence-level information via self-attention. The [CLS] representation captures the contextualized summary of the entire input sequence through interactions with all other tokens in the input.

We obtained 768-dimensional context-aware textual embeddings that encode subtle semantic cues by applying this process to both track and album names. These embeddings were subsequently used as input features for the recommendation model.

Chapter 5

Evaluation and Experiments

5.1 Evaluation

Since our dataset consists of Spotify playlists, it does not contain explicit user feedback such as ratings, likes, or skip behavior. Instead, we treat the inclusion of a song in a playlist as an implicit signal of user preference. If a song appears in a playlist, we assume that the user liked or intended to listen to it. We evaluate the model's ability to retrieve these held-out positive songs based on this assumption.

As described in Section 3.4.2, each playlist is split into training, validation, and test sets, with the test set containing 15 ground-truth songs. To make the evaluation computationally feasible, we restrict the music pool to the set of test songs rather than the entire catalogue. While this reduces the size of the candidate space, it still provides a valid setting for evaluating the model's ranking ability, as it requires identifying relevant songs among a non-trivial number of unseen items.

For each playlist, we compute the hit rate, which measures the proportion of test songs that are successfully retrieved in the top-ranked recommendation list. We use the user embedding derived from the training portion and compute the Pearson similarity between this embedding and all item embeddings in the music pool.

Based on these scores, we retrieve the top 200 most similar songs for each user. A hit is counted if any of the 15 test songs appear in this retrieved list. The final hit rate is computed as the average proportion of test songs that are successfully retrieved across all playlists.

5.2 Comparison of Similarity Measures

To evaluate the impact of similarity computation methods on recommendation performance, we compared Cosine Similarity and Pearson Correlation Coefficient (PCC) under the same model architecture. Specifically, the relevance score between the user embedding and item embedding was computed using each similarity measure.

We conducted this experiment under single feature settings (i.e., audio only, album name only, and track name only). Across all settings, the PCC model consistently achieved higher hit rates than the cosine similarity model. This result indicates

that PCC is more effective for computing user–item relevance in content-based music recommendation scenarios.

Table 5.1: Hit Rate@200 (%) Comparison between Cosine Similarity and Pearson Correlation

Feature Setting	**Cosine Similarity**	**Pearson Correlation**
Audio Only	22.97%	23.63%
Album Name Only	16.90%	18.28%
Track Name Only	18.15%	18.68%

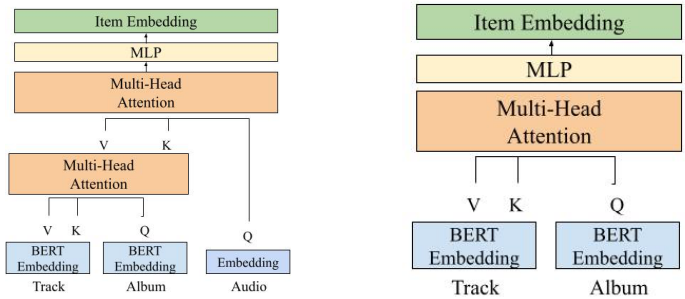
5.3 Text Features Embedding Method Comparison

To compare the representation efficiency of text embedding methods, Word2Vec and BERT, we feed each embedding into a single-feature model using the album name and track name individually. The models showed clear signs of overfitting when the embeddings for album and track names were generated by Word2Vec. The gap between training and validation max-margin losses widened as training progressed, indicating poor generalization. Moreover, the hit rate remained nearly constant across epochs, suggesting that the representations learned by Word2Vec failed to improve actual recommendation performance.

In contrast, replacing these embeddings with BERT-based representations led to a substantial improvement. The hit rate significantly increased when using BERT-based embeddings for either album or track names, and the model converged more stably with reduced overfitting. Furthermore, we observed a noticeable reduction in reconstruction loss when reconstructing the BERT embeddings through the encoder, indicating improved feature representation quality. In comparison, Word2Vec embeddings yielded relatively flat reconstruction loss curves, suggesting limited learning of structural patterns in the input features. These results suggest that BERT embeddings capture richer semantic and contextual information, leading to more effective and generalizable textual representations for recommendation. The training and evaluation trends are illustrated in Appendix Figure 6.5.

5.4 Model Variants Based on Feature Combinations

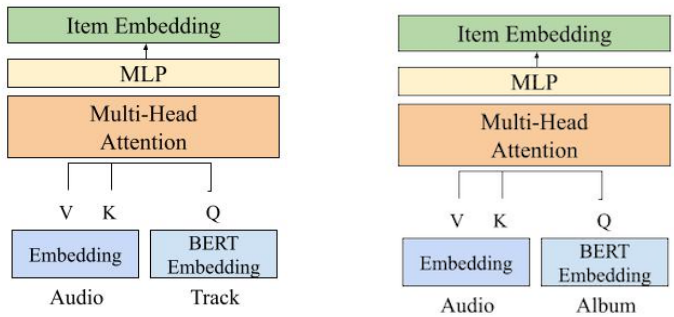
We compare multiple model variants based on different feature combinations. We first evaluate the performance of single-feature models, and then investigate multi-feature models using two fusion strategies: cross-attention fusion and concatenation fusion. For concatenation-based fusion, we empirically evaluated several model configurations and found that a two-layer MLP provided the best balance between model complexity and generalization performance. Adding further layers resulted in overfitting without notable improvements in recommendation accuracy. We further illustrate the design of cross-attention fusion models using various feature combinations. The figures 5.1 and 5.2 show how different embeddings are used as query, key, and value inputs in the attention layers.



(a) Track, album, and audio cross-attention

(b) Track and album cross-attention

Figure 5.1: Architectures of cross-attention fusion models with (a) three features:track, album and audio and (b) two features: track and album



(a) Track,and audio cross-attention

(b) Album and audio cross-attention

Figure 5.2: Architectures of cross-attention fusion models with (a) two feature: track and audio and (b) two features: album and audio

The table 5.1 presents the hit rate@200 results across different feature combinations and fusion types. Among all models, the audio-only model achieved the highest hit rate of 23.63%, outperforming all fusion-based models involving audio features. The track-only model also showed strong performance, reaching a hit rate of 18.68%, which exceeded that of several fusion combinations. Overall, concatenation-based fusion shows better performance than cross-attention fusion. The performance gap was highlighted when combining all three features—album, track, and audio—with concatenation achieving 16.90% and cross-attention only 7.26%. Regarding feature combinations, album + track and audio + track achieved the best results across both fusion strategies. The best combination with the track is different depending on fusion strategies. Specifically, the track combined with album under concatenation (18.42%) and audio under cross-attention (17.69%) yielded the highest performance.

Table 5.2: Model performance comparison across feature combinations (Hit Rate@200, in %).

Fusion Type	**Feature Combination**	**Hit Rate@200 (%)**
Single	Audio Only	23.63
	Album Name Only	16.37
	Track Name Only	18.68
Cross-Attention	Album + Track	16.38
	Album + Audio	14.26
	Track + Audio	17.69
	Album + Track + Audio	7.26
Concatenation	Album + Track	18.42
	Album + Audio	14.72
	Track + Audio	16.04
	Album + Track + Audio	16.90

5.5 Embedding Dimension Size Comparison

Determining the optimal embedding size is crucial to achieving a balance between over-fitting and under-fitting. Furthermore, higher embedding dimensions increase model complexity and computational cost. In models where similarity is computed using the Pearson correlation coefficient between user and item embeddings, a unified and appropriately sized dimension ensures alignment and efficient computation. Ensuring a common dimensionality across all representations helps maintain consistency and comparability during training and evaluation.

To determine an appropriate unified embedding dimension for both user and item representations, we conducted preliminary experiments on single-feature models, using either track or audio features alone. Specifically, we compared performance across different embedding sizes (30, 128, and 256) to identify the optimal dimension for representing both users and items.

Table 5.3: Track-only Model Performance

Dimension	**Hit Rate@200**
30	16.50
128	18.68
256	16.11

Table 5.4: Audio-only Model Performance

Dimension	**Hit Rate@200**
30	17.49
128	23.63

The results show that an embedding size of 128 consistently achieved the best performance in both single-feature settings. Based on this, we adopted a unified embedding dimension of 128 for both user and item embeddings in all subsequent experiments.

5.6 Discussion

5.6.1 Analysis of Similarity Measures

We compared Cosine Similarity and the Pearson Correlation Coefficient (PCC) as similarity measures to compute the relevance score between user and item embeddings. While both are commonly used in recommendation systems, they differ in how they interpret similarity—particularly in their sensitivity to feature-wise scale differences and in how they define vector directionality.

Cosine similarity evaluates the angle between two vectors with respect to the origin, meaning it assesses how aligned the vectors are in a global coordinate space. However, this measure is sensitive to differences in feature scale—a disproportionately large value in a single dimension can dominate the vector's direction and distort similarity estimation, even when the overall trend between vectors is similar. In contrast, PCC normalizes each vector by subtracting its mean and dividing by its standard deviation, thereby centering each vector in its own local space. This standardization removes scale effects and allows PCC to emphasize the relative ordering among features. It captures how similarly two vectors change across dimensions, independent of their scale.

To illustrate, we presented an example where two users demonstrated identical preference trends across musical features (tempo, energy, valence) but with values at vastly different scales:

Feature	**User A**	**User B**
tempo	10	50
energy	2	42
valence	1	41

Both users favored fast, energetic, and positive songs in the same order, yet cosine similarity returned a score of 0.791, as the large tempo value in User B skewed the direction. In contrast, PCC yielded a perfect score of 1.0 because both vectors shared the same relative preference ranking and variation pattern once the scale was normalized. This example highlights that cosine similarity is appropriate when both the direction and relative magnitudes of features carry meaning. In contrast, PCC is better suited for scenarios where the ranking and directional trends of preferences matter more than raw scales—such as in user preference modeling or music recommendation systems, where understanding whether a user consistently prefers one feature over another (e.g., tempo > energy > valence) is more important than their exact rating intensity.

In summary, cosine similarity measures directional alignment from the origin, while PCC evaluates directional consistency after scale normalization. When comparing user-item relationships in contexts where preference flow, relative ranking, and

trend consistency are the focus, PCC provides a more robust and interpretable similarity signal.

5.6.2 Analysis of Text Embedding Techniques

The superior performance of BERT-based embeddings over Word2Vec can be explained by BERT's ability to capture contextual meaning more effectively. Unlike Word2Vec, which produces static embeddings based on limited local context, BERT generates dynamic embeddings by jointly attending to both the left and right contexts of each token. This bidirectional and context-aware encoding allows the model to disambiguate word meanings depending on usage, which is particularly important for interpreting short and often ambiguous metadata such as album or track names.

As a result of this context sensitivity, BERT embeddings tend to be semantically richer and more expressive. In music recommendation, understanding the subtle nuances and semantic relationships between words is critical. Capturing the context awareness meaning and tone of textual metadata improve recommendation quaility. Therefore, BERT-based representations are more suitable than Word2vec for capturing the expressive power of textual features in music recommendation systems such as album and track names, enabling models to learn user preferences more effectively.

5.6.3 Analysis of Model Variants Based on Feature Combinations

Overall, the single-feature models achieved better performance than multi-feature models. In particular, the audio-only model yielded the best result with a hit rate of 23.63%, while combining it with either album or track features led to performance degradation. This may be attributed to the fact that the audio feature set used in this study is highly informative, as it captures a broad range of perceptual and structural characteristics of music. These include rhythmic and dynamic elements (e.g., danceability, energy, tempo), tonal and harmonic properties (e.g., key, mode), acoustic descriptors (e.g., acousticness, instrumentalness), and even contextual or emotional indicators (e.g., valence, liveness, speechiness). By encompassing both low-level and high-level musical attributes, the audio feature set offers rich and multidimensional information about each track. Therefore, the superior performance of the audio-only model may be due to the richness of these features compared to simpler inputs like album or track names. Moreover, combining different modalities—such as audio and text—often requires more complex modeling to capture their interactions. Given the limited size of the dataset, applying such complex fusion methods such as cross-attention may result in suboptimal performance due to insufficient training data.

Regarding fusion strategies, concatenation-based fusion generally outperformed cross-attention fusion, with the exception of the track + audio model. This pattern may be influenced by both the modality type and the generality of the feature. For instance, the album + track combination performed better under concatenation (18.42%) than under cross-attention (16.38%). Since both album and track

names are textual inputs and often reflect similar thematic content, a simple MLP-based concatenation may effectively model their relationship while reducing the risk of overfitting, which can occur when overly complex models are applied to small datasets. Indeed, applying cross-attention fusion resulted in a reduced performance of 16.38%.

In contrast, for models involving audio features—thus combining different modalities—the optimal fusion method varied depending on whether audio was combined with album or track features. The album + audio model performed slightly better under concatenation (14.72%) than cross-attention (14.26%), whereas the track + audio model achieved higher performance with cross-attention (17.69%) compared to concatenation (16.04%).This may be explained by the difference in generality between album and track names. Album names are typically chosen to reflect the overall theme or mood of a collection of tracks, making them relatively more general. In contrast, track names tend to be more specific and diverse, capturing detailed or unique aspects of individual songs. As a result, capturing meaningful interactions between audio and track names—which are less general—may benefit from more expressive fusion methods like cross-attention. On the other hand, the interaction between audio and more general album names may be sufficiently modeled by simpler fusion methods like concatenation, especially in data-constrained settings.

As noted, track names are more detailed and information-rich than album names, which are generally broader and more thematic.This aligns with the experimental result where the track-only model outperformed the album-only model (18.68% vs. 16.37%). Furthermore, since track and album names share the same modality (text), their combination—especially under concatenation fusion—outperformed the track-only model, suggesting that modest increases in input diversity within the same modality can still be beneficial.

The sharp performance drop observed when combining album, track, and audio features under cross-attention fusion can be attributed to the structural and representational limitations of the attention mechanism. Cross-attention requires learning explicit interactions between modalities through a fixed query–key-value structure. In our design, the audio embedding serves as the query, while album and track embeddings serve as keys and values. Although this configuration allows the audio representation to be enriched by contextual cues from textual metadata, it also introduces representational asymmetry. The semantic and structural disparity between continuous audio features and BERT-based textual embeddings can hinder stable alignment within the attention mechanism. Moreover, the model must learn multiple pairwise relationships simultaneously, significantly increasing complexity and introducing potential conflicts—especially when the album and track embeddings already contain overlapping information.

In contrast, concatenation-based fusion bypasses the need for explicit cross-modal alignment. By simply merging features into a shared representation space, the model can implicitly learn their interactions through MLP layers without being constrained by rigid attention roles or weight distributions. This structure is more

robust to feature redundancy and modality heterogeneity and generally performs better in low-resource settings where model stability is critical.

5.6.4 Analysis of Embedding Dimension Selection

The results suggest that an embedding dimension of 128 effectively balances model capacity and generalization. When the embedding size is too small (e.g., 30), the model lacks sufficient capacity to encode the semantic complexity of user preferences and item features. In this case, the limited degrees of freedom restrict the ability to form meaningful latent representations, leading to underfitting and reduced performance. In contrast, when the embedding size is too big, we observed that increasing the embedding size to 256 led to a decline in performance in the track-only model, suggesting that overly large embeddings may cause overfitting, especially when the input feature is relatively simple.

This phenomenon reflects the classic bias-variance trade-off where smaller embeddings introduce high bias by oversimplifying relationships in the data, whereas larger embeddings increase variance as they tend to fit noise rather than underlying patterns. Therefore, selecting an embedding size that achieves an optimal balance between these two extremes is critical for generalization.

In this context, 128-dimensional embeddings offered sufficient expressive power to capture relevant patterns in both track and audio features without incurring the cost of overfitting or inefficiency. Therefore, we adopted 128 as the unified embedding dimension for both user and item representations in our final model.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

With the rise of digital streaming platforms, users now access music conveniently but face choice overload due to vast song libraries. Personalized music recommendation systems have thus become essential in helping users efficiently discover music aligned with their preferences or moods.

To address this challenge, we propose an end-to-end model that jointly learns user and item representations without separate training phases. We extend previous single-modality approaches by incorporating multimodal features—audio, track name, and album name—and systematically comparing fusion strategies, such as concatenation and cross-attention, to capture the complementary strengths of diverse inputs. We further enhance the model by incorporating (1) BERT-based embeddings for contextual metadata representations, (2) lightweight audio features for efficient representation, and (3) a playlist-based binary preference signal to better reflect intentional user behavior.

Experimental results show that audio features are the most informative, followed by track and album features. In most cases, concatenation-based fusion outperformed cross-attention, except when combining track and audio, where their distinct and complementary nature benefited from more complex interaction modeling. The audio + track name and album name + track name emerged as the most effective pairs among all fusion strategies.

We use pretrained BERT model to represent track and album names in a context-aware manner, capturing subtle semantic cues. Compared to Word2Vec, which treats each word individually, BERT considers the surrounding context of each word, allowing for richer and more nuanced representations. This contextual understanding contributed to improved recommendation performance in our experiments.

We also observe that the Pearson Correlation Coefficient (PCC) consistently outperforms Cosine Similarity (CS) as a similarity measure between user and item embeddings. While CS calculates the angle between vectors relative to the origin and is sensitive to feature scale, PCC standardizes each vector by centering at the mean, allowing it to reflect directional preference patterns across features better. This makes PCC more suitable for scenarios like music recommendation, where users exhibit consistent but differently scaled preferences.

These findings highlight the importance of using scale-invariant similarity metrics and well-chosen feature combinations in recommendation systems. Our results further suggest that relative preference trends are often more informative than raw magnitudes and that lightweight fusion strategies may suffice when inputs are semantically distinct.

6.2 Future Work

Dataset Refinement

To evaluate whether the dataset exhibits any inherent structure in the feature space, we conducted an simple unsupervised analysis using a non-deep learning approach—k-means clustering—on each feature set individually (i.e., album, track, and audio). The resulting cluster assignments revealed highly imbalanced distributions: many playlists were assigned to clusters with 0% representation, while a small number of clusters concentrated playlists in narrow bands such as 7% or 13%. These extreme skews suggest a lack of global structure in the embedding space.

We hypothesize that this instability stems from the nature of the dataset, which is based on implicit feedback. In particular, playlists are constructed without explicit labels or preference scores, and may reflect highly subjective, eclectic, or context-specific user behavior. Many playlists combine heterogeneous elements—spanning multiple genres, moods, or time periods—resulting in noisy and diffuse feature representations. Consequently, simple clustering methods like k-means, which rely on geometric compactness and global separability, struggle to form meaningful groupings.

Nevertheless, we observed that a subset of playlists with strong internal coherence—such as those reflecting consistent genre, instrumentation, or emotional tone tended to form more distinguishable clusters. For instance, some clusters achieved relatively dominant proportions (e.g., 7% or 13%), indicating that thematic consistency enables stronger geometric alignment in the feature space. These findings imply that, for deep learning-based models, filtering out noisy or arbitrarily composed playlists, or curating data to ensure greater thematic coherence, could lead to more stable feature distributions and ultimately improve the effectiveness of representation learning.

Improving Feature Fusion Strategies

The current model employs concatenation or cross-attention to fuse album, track, and audio features into a unified item embedding. However, cross-attention imposes a fixed query–key-value structure, which can lead to a representational mismatch between modalities—mainly when the query is derived from audio features and the keys/values are derived from text-based embeddings.

To address this, future work can explore reconfiguring query–key–value roles depending on modality or applying pairwise attention schemes. More flexible fusion mechanisms may improve stability and expressiveness, such as gated attention

(which dynamically weights each modality) and self-attention (to model intermodality structure). For example, when multiple modalities such as audio, track name, and album name are used simultaneously, the weight of each modality would vary depending on context. The audio feature might be more informative in some songs, whereas text features such as track and album names may be more informative due to strong semantic cues. Gated attention mechanisms are designed to dynamically adjust the contribution of each modality by assigning learned weights focusing on the most relevant modalities under different circumstances.

In addition, the internal structure of each modality itself, before combining with different modalities, can carry important information. For instance, in a track title like The Sound of Silence, the position and emphasis of each word may influence the meaning. Self-attention allows the model to capture such inter-modality relationships by selectively emphasizing parts of the input sequence that contribute more significantly to its overall representation. Additionally, attention sparsity or feature selection methods could be employed to reduce redundancy among features, particularly between track and album embeddings.

Enhancing Text Feature Representations

The current text features are obtained from fixed pre-trained BERT embeddings and compressed via autoencoders. While effective, this approach may not fully capture music-domain semantics. Furthermore, fixed representations may limit adaptability to dataset-specific context.

Future improvements could involve fine-tuning the pre-trained language model, such as BERT, on music-related textual data, thereby enriching semantic context to album and track names. Adding metadata such as lyrics, genre, or user-generated tags may also enhance contextual understanding. Moreover, employing contrastive learning can allow the model to learn a structured representation space where semantically similar items (e.g., songs from the same artist or genre) are embedded closer together. Contrastive learning trains the model to embed similar samples closer and dissimilar ones farther apart in the representation space. This could improve both recommendation accuracy and robustness, especially in cold-start scenarios.

Enhancing Modeling with Temporal and Preference information

The current user embedding is derived from a fixed-size playlist vector (e.g. 100) passed through a fully connected layer, which lacks temporal and behavioral information. This restricts the model's ability to capture individual user preferences or listening habits, such as time and sequence of music.

Future directions may include modeling user behavior with sequential architectures such as GRU, LSTM, or Transformer encoders, which can capture the temporal dynamics and ordering of songs within playlists. Incorporating user-level features like preferred genres, emotional tone, or skip/like behaviors could further enrich

personalization.

Hybrid Extension with User–Item Interaction
Furthermore, future work could incorporate user-item interaction signals—commonly used in collaborative filtering—into the representation learning process to extend the current content-based framework into a hybrid recommendation model. This can be achieved by enriching user embeddings with features derived from the user's listening history (e.g., aggregated embeddings of previously interacted tracks), while item embeddings can be constructed by combining the item's inherent content features (such as audio or metadata) with co-listening signals obtained from other users who have engaged with the same item.

This reflects the intuition that their individual preferences best capture user identity, whereas item relevance can benefit from patterns observed across the broader user base. For instance, while user embeddings may encode playlist-level content and behavioural history, item embeddings can simultaneously capture the semantic meaning and collaborative context.

These content and interaction signals can be fused within a unified network trained end-to-end under the existing max-margin loss. This enables the model to jointly learn personalized and semantically meaningful user–item representations without requiring separate optimization objectives. This approach effectively transforms the model into a jointly trained hybrid recommender while preserving the original framework's architectural simplicity and learning paradigm.

References

[1] Ba, J. L., Kiros, J. R., Hinton, G. E. (2016). Layer normalization (arXiv:1607.06450). arXiv. https://arxiv.org/abs/1607.06450

[2] Bhojanapalli, S., Yun, C., Rawat, A. S., Reddi, S. J., Kumar, S. (2020). Low-rank bottleneck in multi-head attention models. arXiv preprint arXiv:2002.07028. https://arxiv.org/abs/2002.07028

[3] Chen, C.-W., Lamere, P., Schedl, M., & Zamani, H. (2018). RecSys Challenge 2018: Automatic music playlist continuation. *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys '18)*. Association for Computing Machinery. https://doi.org/10.1145/3240323.3240342

[4] Dai, Y., Gieseke, F., Oehmcke, S., Wu, Y., Barnard, K. (2020). Attentional feature fusion. arXiv preprint arXiv:2009.14082. https://arxiv.org/abs/2009.14082

[5] Deldjoo, Y., Schedl, M., & Knees, P. (2024). Content-driven music recommendation: Evolution, state of the art, and challenges. *Computer Science Review, 51*, 100618. https://doi.org/10.1016/j.cosrev.2024.100618

[6] Devlin, J., Chang, M. W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186. Association for Computational Linguistics. https://doi.org/10.48550/arXiv.1810.04805

[7] Laskar, M. T. R., Chen, C., Johnston, J., Fu, X.-Y., Bhushan, S., Corston-Oliver, S. (2022). An auto encoder-based dimensionality reduction technique for efficient entity linking in business phone conversations. In Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22) (pp. 3363–3367). ACM. https://doi.org/10.1145/3477495.3536322

[8] Lee, J., Lee, K., Park, J., Park, J., and Nam, J. (2018). Deep Content-User Embedding Model for Music Recommendation. *Proceedings of DLRS 2018*, p. 5.

[9] Mangai, U.G., Samanta, S., Das, S., Chowdhury, P.R. (2010). A survey of decision fusion and feature fusion strategies for pattern classification. IETE Technical Review, 27(4), 293–307.

[10] Michael D. Ekstrand; John T. Riedl; Joseph A. Konstan, Collaborative Filtering Recommender Systems , now, 2011, doi: 10.1561/1100000009. keywords: General Computer Science (Human Computer Interaction);Artificial Intelligence;Machine Learning

[11] Murel, J., Kavlakoglu, E. (2024, March 21). Content-based filtering. IBM. https://www.ibm.com/think/topics/content-based-filtering

[12] Music Technology Group, Music Computing Lab, & The Open University. (2012). Semantic audio content-based music recommendation and visualization based on user preference examples. *Information Processing and Management* . http://dx.doi.org/10.1016/j.ipm.2012.06.004

[13] Praveen, R. G., de Melo, W. C., Ullah, N., Aslam, H., Zeeshan, O., Denorme, T., Pedersoli, M., Koerich, A. L., Bacon, S., Cardinal, P., Granger, E. (2022). A joint cross-attention model for audio-visual fusion in dimensional emotion recognition. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2022, 2486–2495.

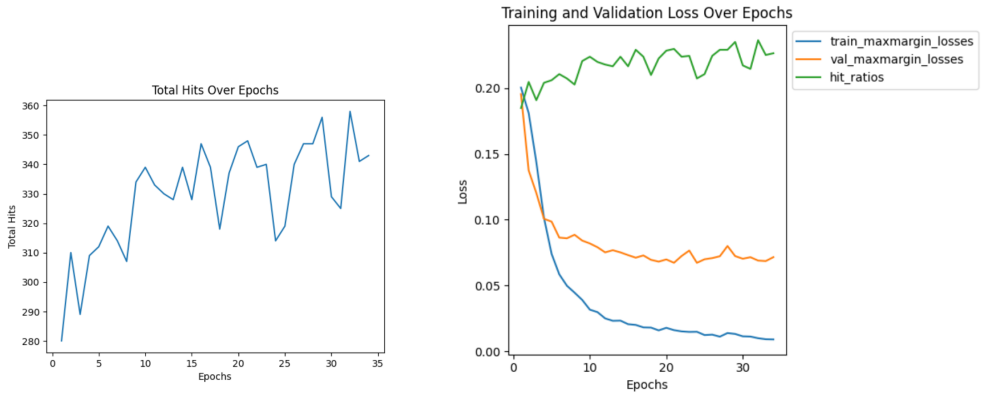
[14] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (Vol. 30).

[15] Wang, Y., Yao, H., Zhao, S. (2015). Auto-encoder based dimensionality reduction. Neurocomputing, 184, 232–242. https://doi.org/10.1016/j.neucom.2015.08.104

[16] Wang, H., Li, J., Wu, H., Hovy, E., Sun, Y. (2022). Pre-trained language models and their applications. Engineering, 25, 51–65. Elsevier. https://doi.org/10.1016/j.eng.2022.04.024

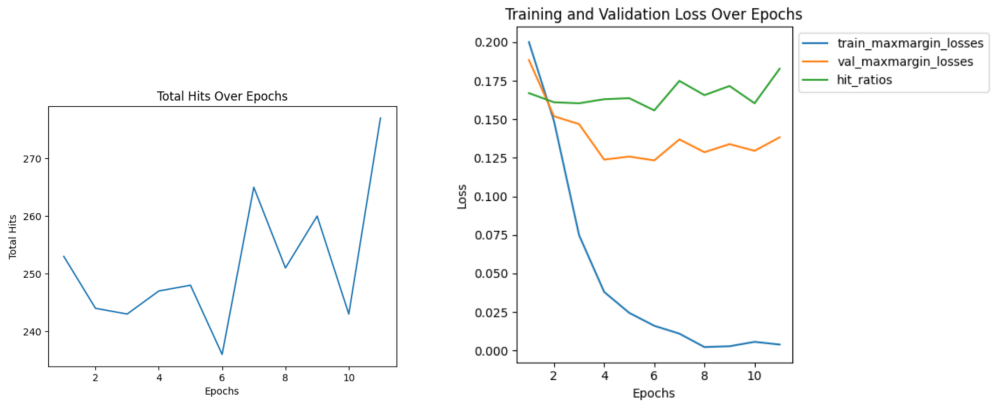
[17] Zeng, Z., Wang, Y., Zhao, Y., and Shi, W., A survey of music recommendation systems, *5th International Conference on Computer Information and Big Data Applications (CIBDA 2024)*, 13 (2024). https://doi.org/10.1145/3671151.3671243

Appendix



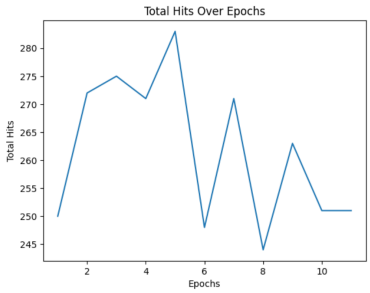
(a) Hit Ratio over Epochs (b) Training Loss over Epochs

Figure 6.1: Performance trends of the audio-only model: (a) hit ratio and (b) training loss over epochs

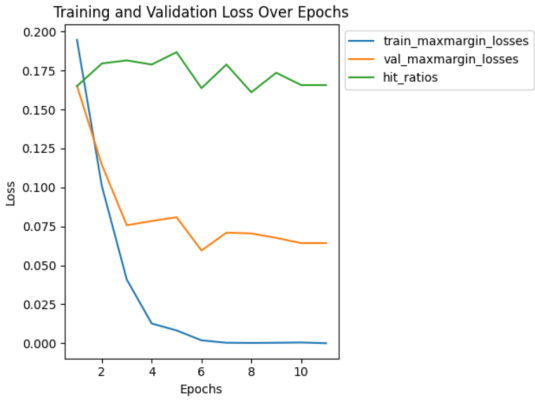


(a) Hit Ratio over Epochs (b) Training Loss over Epochs

Figure 6.2: Performance trends of the album-only model: (a) hit ratio and (b) training loss over epochs

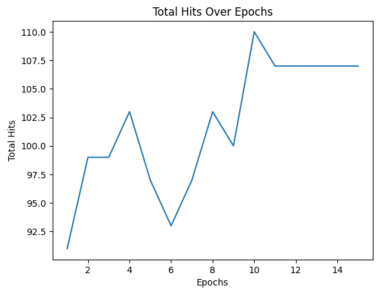


(a) Hit Ratio over Epochs

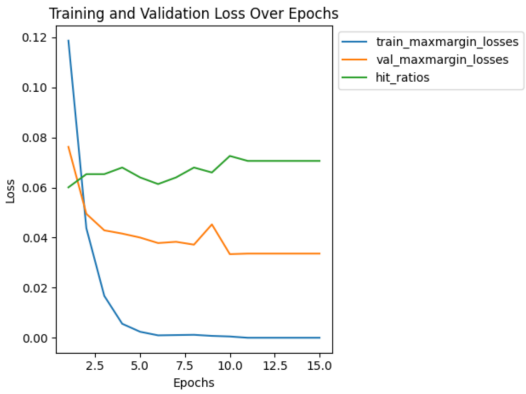


(b) Training Loss over Epochs

Figure 6.3: Performance trends of the track-only model: (a) hit ratio and (b) training loss over epochs

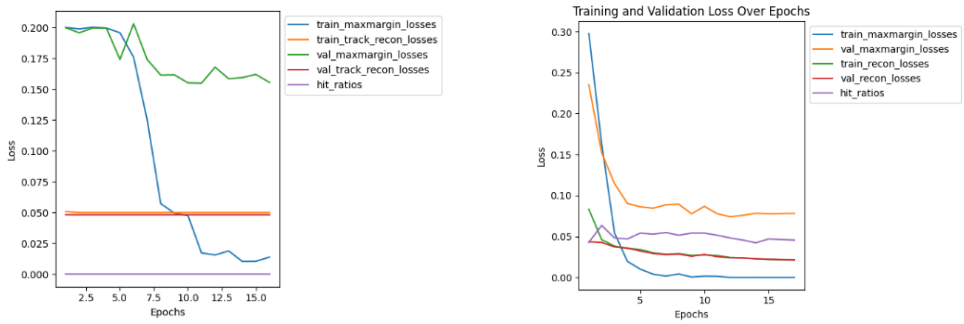


(a) Hit Ratio over Epochs



(b) Training Loss over Epochs

Figure 6.4: Performance trends of the joint of track name, album name, and audio features model: (a) hit ratio and (b) training loss over epochs



(a) Word2Vec (b) BERT

Figure 6.5: Training dynamics and hit ratio trends of the joint track and album embedding model using (a) Word2Vec and (b) BERT representations